JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 09 OF 17 - STUDY STEP 09

Recursion and Backtracking in Java

Contracts, Call Stacks, Search Trees, Pruning, and State Restoration

BY

Vinay Reddy Kalluri

Reason from recursive contracts, understand stack cost, restore mutable state correctly, and prune combinatorial search without losing solutions.

RECURSION | STACK FRAMES | BACKTRACKING | PRUNING | SEARCH TREES

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `DSA 08 — Hashing and Prefix State`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume separates recursion as a method contract from backtracking as controlled state-space search, with Java stack and mutation behavior made explicit.

Readiness map

Decision	Evidence
START HERE	The problem decomposes into smaller instances of the same contract; A decision tree must enumerate valid combinations; State must be chosen, explored, and undone; Input depth can threaten the Java stack.
PREPARE FIRST	Study Steps 01-08; Complexity and collection fundamentals.
MOVE ON WHEN	Write precise recursive contracts and base cases; Analyze call depth and output-sensitive work; Implement choose-explore-unchoose safely; Apply pruning without invalidating completeness.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 08 - Hashing and Prefix State	**YOU ARE HERE** **DSA 09 - Recursion and Backtracking**	DSA 10 - Linked Lists

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Recursive Contracts and Backtracking Patterns for SDE-2

Recursion is not a trick for making a loop disappear. It is a way to delegate a smaller instance of a problem under a precise contract. Backtracking adds a second idea: make one reversible decision, explore everything below it, then restore the caller's state. At SDE-2 level, an answer is incomplete if it merely produces the right combinations. You should be able to name the state, prove that every legal answer is generated exactly as intended, bound the search tree, and explain who owns each mutable object.

The pattern map

Signal in the prompt	State to carry	Typical pattern
"Generate every subset"	next index and current selection	include/exclude or start-index DFS
"Choose k from n"	next candidate, choices remaining	combinations with a feasibility bound
"Use values to reach a target"	start index and remainder	combination search with numeric pruning
"Every ordering"	used positions and current order	permutation search
"Find a path spelling a word"	board coordinate and word offset	grid DFS with temporary visitation
"Place items without conflicts"	row and occupied constraints	constraint backtracking, often bit/set state
"Count/optimize, answers overlap"	arguments that fully identify a subproblem	memoization or dynamic programming

The recognition distinction matters. A traversal follows edges already present in the input. Backtracking constructs a decision tree that is usually not stored anywhere. Memoization caches the result of a state only when that result is independent of the path used to reach it.

Start with a recursive contract

For every helper, say one complete sentence before coding:

> `search(i, path)` appends every valid completion whose decisions before `i` are exactly `path`, and leaves `path` unchanged when it returns.

That sentence gives four proof obligations.

1. **Meaning:** the parameters describe all information needed below this call.
2. **Base case:** a smallest state returns the correct result without another call.
3. **Progress:** every recursive edge moves toward a base case.
4. **Preservation:** mutable state observed by the caller is restored before return.

A recursion proof is structural induction. Assume calls on smaller states satisfy the contract. Show that the current call partitions its legal answers into the recursive branches, combines their results correctly, and terminates. The code should mirror that proof.

Stack and allocation reality in Java

Each ordinary Java call consumes a stack frame. Java does not guarantee tail-call elimination, so a logically tail-recursive method can still overflow the stack. Depth `O(log n)` is normally comfortable; depth proportional to an untrusted input may not be. A million-node chain, a long flood fill, or a degenerate tree should trigger an iterative design or an explicit stack.

Allocation is separate from recursion depth. A `new ArrayList<>(path)` at every leaf is required when returning snapshots; returning the live `path` would alias one object and corrupt all results. Copying at every internal node, however, can add unnecessary allocation. Prefer one owned mutable path, mutate/recurse/undo, and copy only when publishing an answer.

Backtracking as a transaction

The invariant is:

> On entry to a call, shared mutable state represents exactly the decisions on the path from the root to this node; on exit, it is byte-for-byte or logically equivalent to its entry state.

Use a transaction-shaped sequence:

1. choose;
2. mutate the owned search state;
3. recurse;
4. undo in the reverse order.

If the recursive call can throw and the state must remain reusable, production code may need `try/finally` around the undo. Interview problems normally assume no exception inside the search, but naming the issue demonstrates ownership awareness.

Family 1: subsets

Recognition and invariant

Use subset DFS when each element may be selected at most once and output order does not matter. With a start-index formulation, `start` is the first undecided position. `path` contains an increasing sequence of positions, so no subset is generated twice. At every call, the current path is itself a valid subset and can be emitted.

TRACE IT | Dry run

For `[1, 2]`, emit `[]`. Choose index `0`: emit `[1]`; choose index `1`: emit `[1,2]`; undo `2`, undo `1`. Then choose index `1`: emit `[2]`. The answer sequence is `[], [1], [1,2], [2]`. More important than this order is the restoration: after the child for `[1,2]` returns, the caller again owns `[1]`.

Correctness and cost

Every subset has one unique increasing list of indexes. The loop follows exactly that list, so it reaches every subset once. There are `2^n` answers and copying each can cost `O(n)`, giving `O(n * 2^n)` output time and `O(n)` search depth excluding output. No algorithm can return all subsets in sub-output-size time.

Edges and mistakes

The empty input has one subset: the empty subset. If equal input values should not create duplicate value-subsets, sort first and skip equal candidates at the same depth; the plain implementation treats positions as distinct. Do not append `path` itself to results. Append a snapshot.

Family 2: fixed-size combinations and combination sum

Recognition and pruning

Combinations choose without regard to order. In `choose k from 1..n`, carry the next legal number. A useful bound is: if `need` values remain, the largest legal starting value is `n - need + 1`. Iterating past it creates calls that cannot possibly finish.

For combination sum with reusable positive candidates, sort and carry a `start` index. Reusing a candidate calls the child with the same index; disallowing reuse calls with `i + 1`. Positivity makes `candidate > remainder` a sound stopping rule. That prune is invalid if negative or zero candidates are allowed, so the API contract must reject them or use a different state model.

Invariant and dry run

For candidates `[2,3,6,7]` and target `7`, the path is nondecreasing because future choices begin at `start`. From `[]`, choose `2`, then `2`, then `2`; remainder `1` prunes all choices. Restore to `[2,2]`, then try `3`, reaching remainder `0` and publishing `[2,2,3]`. Later the root chooses `7` and publishes `[7]`. Ordering plus same-depth duplicate skipping prevents permutation duplicates such as `[2,3,2]`.

The invariant is that `sum(path) + remainder` equals the original target, every element in `path` came from the candidate set, and path indexes are nondecreasing. A remainder of zero is therefore a valid answer. With positive candidates, every recursive call reduces the remainder, proving termination.

Complexity

Combination generation costs at least the total output size. A coarse search bound for choose-`k` is `O(C(n,k) * k)`. Combination sum can have exponentially many nodes; with minimum value `m`, depth is at most `target / m`. State the output-sensitive reality instead of claiming a misleading single polynomial bound.

Family 3: unique permutations

Recognition and invariant

Permutations care about order, so a start index is insufficient. Carry a `used[]` flag for positions. At depth `d`, exactly `d` positions are used and `path` is their ordered sequence. Sort the input. At one depth, skip `values[i]` when it equals `values[i-1]` and the earlier equal position has not been used. This rule chooses one representative among indistinguishable siblings while still allowing equal values at different depths.

For `[1,1,2]`, the root may choose the first `1`, but it skips the second `1` as an equivalent root branch. Below the first `1`, the second can be chosen because its predecessor is already used. The leaves are `[1,1,2]`, `[1,2,1]`, and `[2,1,1]`.

There are at most `n!` leaves and each snapshot costs `O(n)`, so the upper bound is `O(n * n!)` time and `O(n)` auxiliary search state, excluding results. Sorting adds `O(n log n)`.

Family 4: grid Word Search

Recognition and ownership

Use grid backtracking when one path must satisfy an ordered constraint and a cell cannot be reused within that path. The state is `(row, column, offset)` plus visitation. Four neighbors form the branches. A separate visited matrix accepts every `char` value safely. In-place marking is an optional optimization only when a sentinel is provably outside the input alphabet and every return path restores the board.

The helper contract is: starting at this cell with all earlier path cells marked visited, report whether the suffix beginning at `offset` can be matched, and restore the visited state before return. Bounds, prior visitation, and character mismatch reject a state. Matching the last character succeeds.

TRACE IT | Dry run and proof

On board rows `ABCE`, `SFCS`, `ADEE`, searching `ABCCED` starts at `A(0,0)`, advances right to `B`, right to `C`, down to `C`, left to `E`, and left to `D`. Each entered cell is temporarily marked in the visited matrix, so the path cannot turn back and reuse a prior cell. On unwind, every visitation mark is restored.

The branching factor is at most four for the first step and at most three afterward because the previous cell is marked. A safe bound is `O(rows * cols * 4^L)` for word length `L`, with `O(L)` call depth. Useful production prunes include rejecting when `L` exceeds the cell count and starting from the rarer endpoint after comparing board frequencies. Do not silently reverse the caller's word; keep that optimization internal.

Family 5: N-Queens and constraint state

Recognition

N-Queens represents the broader family "place one decision per level while constraints accumulate." Place one queen per row. Then the state needs only occupied columns, descending diagonals `row - col`, and ascending diagonals `row + col`. Sets make the invariant explicit; bit masks are a later optimization when `n` is safely bounded by the integer width.

At row `r`, rows `[0,r)` contain exactly one queen and none attack one another. Trying column `c` is legal precisely when its column and both diagonal keys are absent. Mark all three constraints, recurse to `r+1`, then unmark them. When `r == n`, the invariant proves a complete legal board.

For `n=4`, choosing row positions `[1,3,0,2]` yields `.Q..`, `...Q`, `Q...`, `..Q.`. A branch beginning with column `0` eventually reaches a row with no legal column and backtracks. This failure is information about that branch only; no global "impossible" cache is sound unless the complete constraint state is part of the key.

The worst case is conventionally bounded by `O(n!)` candidate placements, with `O(n)` depth and `O(n)` constraint/path state excluding output. Symmetry breaking can halve root work, but it complicates output enumeration and should be introduced only when the contract permits reconstructing mirrored boards.

Where memoization begins-and where it does not

Memoization is sound when a function's result depends only on its cache key. Counting ways to climb from step `i` depends only on `i`, so cache `i`. A Word Search call that uses `(r,c,offset)` but omits which cells are already visited is not the same subproblem; caching `false` under that incomplete key can reject a valid path. Likewise, N-Queens needs row plus occupied constraints, not row alone.

Ask two questions:

1. If two paths reach the same proposed key, are their legal future choices identical?
2. Is the returned value independent of result-list side effects and caller-owned mutable state?

If either answer is no, expand the key, copy/freeze the relevant state, or do not memoize. Backtracking enumerates path-dependent possibilities; dynamic programming merges truly equivalent states.

Complete Java 21 reference implementation

The class below is intentionally standalone. Every public entry point validates the assumptions on which its pruning depends, and `main` uses assertions as an executable review checklist. Run it with `java -ea RecursionBacktrackingSde2`.

JAVA (continued 1/7)

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public final class RecursionBacktrackingSde2 {
    private RecursionBacktrackingSde2() {}

    public static List<List<Integer>> subsets(int[] values) {
        if (values == null) throw new IllegalArgumentException("values is null");
        List<List<Integer>> answer = new ArrayList<>();
        subsetsDfs(values, 0, new ArrayList<>(), answer);
        return answer;
    }

    private static void subsetsDfs(int[] a, int start, List<Integer> path,
                                   List<List<Integer>> answer) {
        answer.add(new ArrayList<>(path));
        for (int i = start; i < a.length; i++) {
            path.add(a[i]);
            subsetsDfs(a, i + 1, path, answer);
            path.remove(path.size() - 1);
        }
    }

    public static List<List<Integer>> combine(int n, int k) {
        if (n < 0 || k < 0 || k > n) return List.of();
        List<List<Integer>> answer = new ArrayList<>();
        combineDfs(1, n, k, new ArrayList<>(), answer);
        return answer;
    }
```

JAVA (continued 2/7)

```
    private static void combineDfs(int start, int n, int need,
                                   List<Integer> path,
                                   List<List<Integer>> answer) {
        if (need == 0) {
            answer.add(new ArrayList<>(path));
            return;
        }
        for (int value = start; value <= n - need + 1; value++) {
            path.add(value);
            combineDfs(value + 1, n, need - 1, path, answer);
            path.remove(path.size() - 1);
        }
    }

    public static List<List<Integer>> combinationSum(int[] candidates,
                                                      int target) {
        if (candidates == null || target < 0) {
            throw new IllegalArgumentException("invalid input");
        }
        int[] sorted = candidates.clone();
        Arrays.sort(sorted);
        for (int value : sorted) {
            if (value <= 0) {
                throw new IllegalArgumentException("candidates must be positive");
            }
        }
        List<List<Integer>> answer = new ArrayList<>();
        sumDfs(sorted, 0, target, new ArrayList<>(), answer);
        return answer;
    }

    private static void sumDfs(int[] a, int start, int remaining,
                               List<Integer> path,
```

JAVA (continued 3/7)

```
                            List<List<Integer>> answer) {
        if (remaining == 0) {
            answer.add(new ArrayList<>(path));
            return;
        }
        for (int i = start; i < a.length && a[i] <= remaining; i++) {
            if (i > start && a[i] == a[i - 1]) continue;
            path.add(a[i]);
            sumDfs(a, i, remaining - a[i], path, answer);
            path.remove(path.size() - 1);
        }
    }

    public static List<List<Integer>> permuteUnique(int[] values) {
        if (values == null) throw new IllegalArgumentException("values is null");
        int[] sorted = values.clone();
        Arrays.sort(sorted);
        List<List<Integer>> answer = new ArrayList<>();
        permuteDfs(sorted, new boolean[sorted.length],
                   new ArrayList<>(), answer);
        return answer;
    }

    private static void permuteDfs(int[] a, boolean[] used,
                                   List<Integer> path,
                                   List<List<Integer>> answer) {
        if (path.size() == a.length) {
            answer.add(new ArrayList<>(path));
            return;
        }
        for (int i = 0; i < a.length; i++) {
            if (used[i]) continue;
```

JAVA (continued 4/7)

```
            if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;
            used[i] = true;
            path.add(a[i]);
            permuteDfs(a, used, path, answer);
            path.remove(path.size() - 1);
            used[i] = false;
        }
    }

    public static boolean exists(char[][] board, String word) {
        if (board == null || word == null) {
            throw new IllegalArgumentException("null input");
        }
        if (word.isEmpty()) return true;
        long cells = 0;
        boolean[][] visited = new boolean[board.length][];
        for (int r = 0; r < board.length; r++) {
            char[] row = board[r];
            if (row == null) throw new IllegalArgumentException("null row");
            cells += row.length;
            visited[r] = new boolean[row.length];
        }
        if (word.length() > cells) return false;
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[r].length; c++) {
                if (wordDfs(board, visited, r, c, word, 0)) return true;
            }
        }
        return false;
    }

    private static boolean wordDfs(char[][] b, boolean[][] visited, int r, int c,
```

JAVA (continued 5/7)

```
                                String word, int offset) {
        if (r < 0 || r >= b.length || c < 0 || c >= b[r].length
                || visited[r][c] || b[r][c] != word.charAt(offset)) return false;
        if (offset == word.length() - 1) return true;
        visited[r][c] = true;
        boolean found = wordDfs(b, visited, r - 1, c, word, offset + 1)
                || wordDfs(b, visited, r + 1, c, word, offset + 1)
                || wordDfs(b, visited, r, c - 1, word, offset + 1)
                || wordDfs(b, visited, r, c + 1, word, offset + 1);
        visited[r][c] = false;
        return found;
    }

    public static List<List<String>> solveNQueens(int n) {
        if (n < 0) throw new IllegalArgumentException("negative n");
        List<List<String>> answer = new ArrayList<>();
        int[] columnAtRow = new int[n];
        boolean[] columns = new boolean[n];
        boolean[] descending = new boolean[Math.max(0, 2 * n - 1)];
        boolean[] ascending = new boolean[Math.max(0, 2 * n - 1)];
        queensDfs(0, n, columnAtRow, columns, descending, ascending, answer);
        return answer;
    }

    private static void queensDfs(int row, int n, int[] positions,
                                  boolean[] columns, boolean[] descending,
                                  boolean[] ascending,
                                  List<List<String>> answer) {
        if (row == n) {
            List<String> board = new ArrayList<>(n);
            for (int r = 0; r < n; r++) {
                char[] line = new char[n];
```

JAVA (continued 6/7)

```
                Arrays.fill(line, '.');
                line[positions[r]] = 'Q';
                board.add(new String(line));
            }
            answer.add(board);
            return;
        }
        for (int col = 0; col < n; col++) {
            int down = row - col + n - 1;
            int up = row + col;
            if (columns[col] || descending[down] || ascending[up]) continue;
            positions[row] = col;
            columns[col] = descending[down] = ascending[up] = true;
            queensDfs(row + 1, n, positions, columns,
                      descending, ascending, answer);
            columns[col] = descending[down] = ascending[up] = false;
        }
    }

    public static long countClimbs(int steps) {
        if (steps < 0) return 0;
        if (steps > 91) throw new ArithmeticException("result exceeds long");
        long[] memo = new long[steps + 1];
        Arrays.fill(memo, -1);
        return countClimbs(steps, memo);
    }

    private static long countClimbs(int remaining, long[] memo) {
        if (remaining == 0) return 1;
        if (remaining < 0) return 0;
        if (memo[remaining] != -1) return memo[remaining];
        return memo[remaining] = Math.addExact(
```

JAVA (continued 7/7)

```
                countClimbs(remaining - 1, memo),
                countClimbs(remaining - 2, memo));
    }

    public static void main(String[] args) {
        assert subsets(new int[] {1, 2, 3}).size() == 8;
        assert combine(4, 2).size() == 6;
        assert combinationSum(new int[] {2, 3, 6, 7}, 7).size() == 2;
        assert permuteUnique(new int[] {1, 1, 2}).size() == 3;

        char[][] board = {
            {'A', 'B', 'C', 'E'},
            {'S', 'F', 'C', 'S'},
            {'A', 'D', 'E', 'E'}
        };
        char[][] before = Arrays.stream(board).map(char[]::clone)
                                .toArray(char[][]::new);
        assert exists(board, "ABCCED");
        assert !exists(board, "ABCB");
        assert !exists(new char[][] {{'A', 'B', 'X'}}, "AB\0");
        assert Arrays.deepEquals(board, before) : "board must be restored";
        assert solveNQueens(4).size() == 2;
        assert countClimbs(5) == 8;
        boolean overflowRejected = false;
        try {
            countClimbs(92);
        } catch (ArithmeticException expected) {
            overflowRejected = true;
        }
        assert overflowRejected;
    }
}
```

Interview-grade edge-case checklist

- Empty choices: subsets returns one empty answer; permutations likewise have one mathematical empty ordering, depending on the API contract.
- Duplicate values: decide whether positions or values define uniqueness. Sort-and-skip only when value uniqueness is required.
- Invalid numeric domains: positivity is part of combination-sum termination and pruning.
- Mutable input: document temporary mutation and restoration when using it. The sample uses call-owned visitation and does not mutate the board, but callers must still keep the input stable while a search is running.
- Ragged grids: horizontal bounds belong to the current row, not row zero. Vertical moves into a different-length row require checking that destination row's width.
- Sentinel collisions: an in-place alternative that marks with `\0` is correct only when that value is outside the input alphabet. The sample's `boolean[][]` removes that assumption at an allocation cost.
- Exponential output: do not promise to materialize an unbounded answer set. A production API might expose an iterator, callback, limit, deadline, or cancellation token.
- Integer overflow: counts can overflow `long` long before the search becomes practically enumerable. The sample rejects climb counts beyond the exact `long` range; another API may use `BigInteger`, saturation, or a documented modulus.
- Stack depth: translate input constraints into maximum depth before choosing recursion.

TRY IT | Exercises with model checkpoints

Exercise 1: duplicate subsets

Return unique subsets of an integer array that may contain duplicates.

Checkpoint: sort a defensive copy; at each depth skip `a[i]` when `i > start && a[i] == a[i-1]`. Do not globally skip duplicates, because two equal values may both appear in one subset. Your invariant should still use increasing positions. Expected complexity is output-sensitive and bounded above by `O(n * 2^n)`.

Exercise 2: combinations without materializing

Expose combinations through `Consumer<List<Integer>>` and stop after a caller-provided limit.

Checkpoint: publish an immutable snapshot or clearly scoped read-only view. Propagate a boolean "stop" result upward so all frames terminate promptly. Validate a nonnegative limit. Discuss whether callbacks may re-enter the generator or throw.

Exercise 3: restore under failure

Modify Word Search so the matching predicate may throw.

Checkpoint: save the character, mark it, and place recursive exploration in `try` with restoration in `finally`. The method's ownership contract then survives both normal and exceptional exits.

Exercise 4: palindrome partitioning

Partition a string into all lists of palindromic substrings.

Checkpoint: the state is a start offset; branches choose an end offset whose slice is a palindrome. Precomputing palindrome truth for all intervals costs `O(n^2)` and avoids rescanning each slice. Copy the path only at `start == length`.

Exercise 5: memoization audit

Explain why `(row, col, offset)` is an insufficient Word Search cache key.

Checkpoint: two calls can share those three values but have different cells unavailable because their earlier paths differ. Their future legal transitions are therefore different. A complete visited-set key is possible but often too large; ordinary backtracking is the appropriate baseline.

Exercise 6: N-Queens count only

Return only the number of solutions using bit masks.

Checkpoint: mask available columns as `all & ~(columns | diagLeft | diagRight)`, extract the lowest set bit, recurse, and shift diagonal attack masks. Define the maximum `n` supported by the chosen primitive and use unsigned shifts where needed. Counting avoids board allocation but not the exponential search.

SDE-2 production follow-ups

How would you make an exponential generator safe in a service? Put explicit limits on input size, results, elapsed time, and memory. Stream results when the transport supports backpressure, propagate cancellation, record truncation in the response contract, and meter the work. Avoid holding locks across callbacks.

Can searches be parallelized? Root branches are often independent after their state is copied. Parallelism is useful only when branches are coarse enough to amortize task and allocation overhead. Shared result ordering, cancellation, and output limits become synchronization concerns. A bounded executor is safer than recursively spawning an unbounded task tree.

Would you mutate the caller's board? In a coding interview, temporary restoration is a standard space optimization. In a library, the default should be a clearly documented ownership contract or a private copy. Concurrent callers, observability hooks, and exceptions make invisible mutation risky.

What evidence would you collect? Count expanded nodes, pruned branches, maximum depth, results produced, and cancellation latency. Those metrics distinguish a poor pruning rule from expensive required output. Benchmark representative distributions, not only one friendly input.

Interview follow-up chain and model answers

Why not pass a fresh path copy into every child? It can be correct, and sometimes it is the clearest ownership choice. However, a depth-`d` path copy at every search node adds allocation and copying beyond the unavoidable snapshots at published leaves. A single caller-owned path with strict restoration usually reduces garbage. I would choose based on branch count, path size, concurrency needs, and whether exceptions/callbacks make restoration fragile.

Can you prove the duplicate-permutation skip? After sorting, equal unused values at the same depth create indistinguishable next paths. The rule permits only the first currently unused representative. It does not suppress the later equal value when the earlier one is already in the path, so `[1,1,2]` remains reachable. Thus equivalent sibling subtrees are collapsed while valid repeated values across depths remain.

What changes when only one answer is required? Return a boolean or optional result upward and short-circuit after the first success. Still restore mutable state before returning. Candidate order then affects which answer is returned and latency, so make ordering deterministic if the API or tests depend on it. Complexity remains exponential in the worst case, but favorable ordering may find a solution early.

When would you replace recursion with an explicit stack? When maximum depth is input-dependent and can exceed the runtime stack budget, when a search must pause/resume, or when checkpointing and cancellation require control over frames. Each explicit frame stores the recursive parameters plus which branch should run next. This moves memory to the heap but does not reduce asymptotic state.

How do you review a proposed prune? Write the constraint as a necessary condition for every completion below the current state. Prove the condition cannot become true again after the branch is discarded. For positive combination sum, `candidate > remainder` is final because later sorted candidates are no smaller. The same statement is false with negative values, so carrying that optimization across contracts is a correctness bug.

How would you count without enumerating? Change the helper return contract from "publish every completion" to "return the number of completions." Then equivalent states may become memoizable, and path snapshots disappear. The count may overflow even when enumeration is impossible; select `long`, `BigInteger`, saturation, or modulo according to the caller's contract. Counting and listing are different APIs with different complexity and memory behavior.

Final review checklist

- I can state the helper contract without referring to its implementation.
- I can identify base case, progress measure, and maximum depth.
- I know whether ordering matters and whether duplicates are positional or value-based.
- Every mutation has a paired restoration before every return.
- Pruning follows from an explicit constraint; it is not a guess.
- Memoization keys include all state that changes future choices.
- Complexity includes both search nodes and the size/cost of returned answers.
- The production API has limits, ownership, cancellation, and overflow policies.

That is the standard for recursion at SDE-2: not merely reaching a leaf, but controlling the contract of the entire search tree.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: factorial, tree height, and simple subsets
- Interview Core: permutations, combinations, and grid search
- SDE-2 Follow-up: bound depth and control allocation
- Challenge: design sound pruning rules

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: DSA 08 - Hashing: Maps, Sets, Frequency, and Prefix State

Series index

Next book: DSA 10 - Linked Lists: Pointer Reasoning and Mutation

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, or System Design and Backend first, then follow the books inside that segment in order. Stable study-step codes remain on filenames and links; the clearer segment book codes appear on the website and every PDF cover.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Advanced Java B: Language, OOP, and Modern Java
Java	**JAVA 05**	Advanced Java C: Collections, Streams, and I/O
Java	**JAVA 06**	Advanced Java A: JVM and Execution
Java	**JAVA 07**	Advanced Java D: Concurrency and the Memory Model
Java	**JAVA 08**	Advanced Java E: Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Advanced Java G: Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
System Design	**SD 01**	Advanced Java F: Design, Backend, Testing, and Security
System Design	**SD 02**	MySQL for Java Backend Interviews
System Design	**SD 03**	Hibernate and JPA for Java Backend Interviews
System Design	**SD 04**	Spring Framework for Java Engineers
System Design	**SD 05**	Spring Boot for Java Backend Engineers

SEGMENT	BOOK	FOCUSED LEARNING PATH
System Design	**SD 06**	Spring Data for Java Backend Engineers
System Design	**SD 07**	MongoDB for Java Backend Interviews
System Design	**SD 08**	Redis for Java Backend Interviews
System Design	**SD 09**	Apache Kafka and Spring Kafka
System Design	**SD 10**	Spring Ecosystem Extensions
System Design	**SD 11**	Spring AI for Java Engineers
System Design	**SD 12**	Advanced Java H: Spring Boot and REST APIs
System Design	**SD 13**	Advanced Java I: Persistence, SQL, JPA, and Caching
System Design	**SD 14**	Advanced Java J: Distributed Systems and System Design

Roadmap editions identify planned books whose chapter sets will be expanded one at a time. Existing deep-dive books remain available later in each segment, so no published material is displaced.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 09 of 17 - Study Step 09. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.